FACULDADE DE TECNOLOGIA DE FRANCA

“Dr. THOMAZ NOVELINO”

TECNOLOGIA EM DESENVOLVIMENTO SOFTWARE MULTIPLATAFORMA

ANA JÚLIA ALVES MOTA

LAUANE GABRIELA DE ARAÚJO TOLEDO

PEDRO HENRIQUE CINTRA SILVA

MINERAÇÃO DE DADOS

Relatório: Entrega Final de Mineração de Dados do Projeto Integrador

FRANCA/SP

2026

1. OBJETIVOS E ORIGEM DOS DADOS

O DoeFácil é um sistema multiplataforma voltado para a gestão e organização de campanhas, doações e controle de estoque de instituições sociais. No escopo da disciplina de Mineração de Dados, esta etapa final aprofunda a análise exploratória e a modelagem estatística preditiva e espacial sobre uma base de dados externa de instituições sociais.

A investigação fundamenta-se no arquivo *givefoodbasededados.json*, disponibilizado no repositório do grupo. A origem primária dos registros é a API pública do projeto Give Food, que compila cadastros de bancos de alimentos e entidades assistenciais (GIVE FOOD, s.d.). Cada entrada representa uma instituição formalmente catalogada.

O conjunto contém 1.023 registros e 17 atributos descritivos, englobando identificadores únicos, nomes oficiais, classificações territoriais, situação operacional e geolocalização por latitude e longitude. A base abrange territórios do Reino Unido (Inglaterra, Escócia, País de Gales e Irlanda do Norte) e dependências associadas.

Tabela 1 — Identificação da base e parâmetros de versão

Parâmetro	Especificação do Registro
Fonte Original	Give Food API versão 2
Arquivo de Trabalho	givefoodbasededados.json
Volume Bruto	1.023 cadastros institucionais
Branch de Versão	feat/ana-final-mineracao

2. PREPARAÇÃO DOS DADOS

Para assegurar a integridade e reprodutibilidade analítica, o script mining/tratar.py automatiza a limpeza, tratamento de nulos e conversão de tipos

diretamente sobre a cópia local estática, evitando dependências de requisições web ativas durante a execução dos testes.

2.1 LIMPEZA E FILTRAGEM

O processamento emprega a biblioteca Pandas para converter o conteúdo JSON em estruturas tabulares. Realiza-se a verificação de duplicidades com base na chave id. O tratamento separa o campo combinado de coordenadas (lat_lng) em duas variáveis numéricas distintas (latitude e longitude), aplicando coerção de erros (errors='coerce') para descartar posições corrompidas.

O filtro de elegibilidade considera apenas instituições ativas (closed == False), com strings de nome preenchidas e coordenadas geográficas estritamente confinadas aos limites globais válidos (-90 a 90 para latitude e -180 a 180 para longitude).

3. RESULTADOS DA PREPARAÇÃO

A execução do pipeline de pré-processamento validou integralmente o volume fornecido, gerando o arquivo consolidado *mining/data/instituicoes_tratadas.csv* e o relatório de rastreabilidade contendo o hash criptográfico SHA-256 do snapshot analisado.

Tabela 2 — Resumo quantitativo do tratamento

Etapa de Verificação	**Quantidade**
Registros lidos da base bruta	1.023
Duplicatas por ID removidas	0
Registros inativos/inválidos descartados	0
Total de registros validados e exportados	**1.023**

4. MODELAGEM COM APRENDIZADO NÃO SUPERVISIONADO

A modelagem espacial foi conduzida por meio do algoritmo de agrupamento não supervisionado K-Means. Diferente da análise preliminar fixa anterior, a Sprint Final implementou uma variação sistemática do número de clusters (k) de 2 a 6 utilizando o script *mining/comparar_clusters.py*.

Utilizou-se como variáveis descritivas exclusivas a latitude e a longitude. Para evitar distorções decorrentes da diferença de escalas, os dados foram pré-processados via StandardScaler, padronizando as feições com média zero e desvio padrão unitário. Os parâmetros fixados incluíram random_state=42 e n_init=10 para estabilização dos centroides.

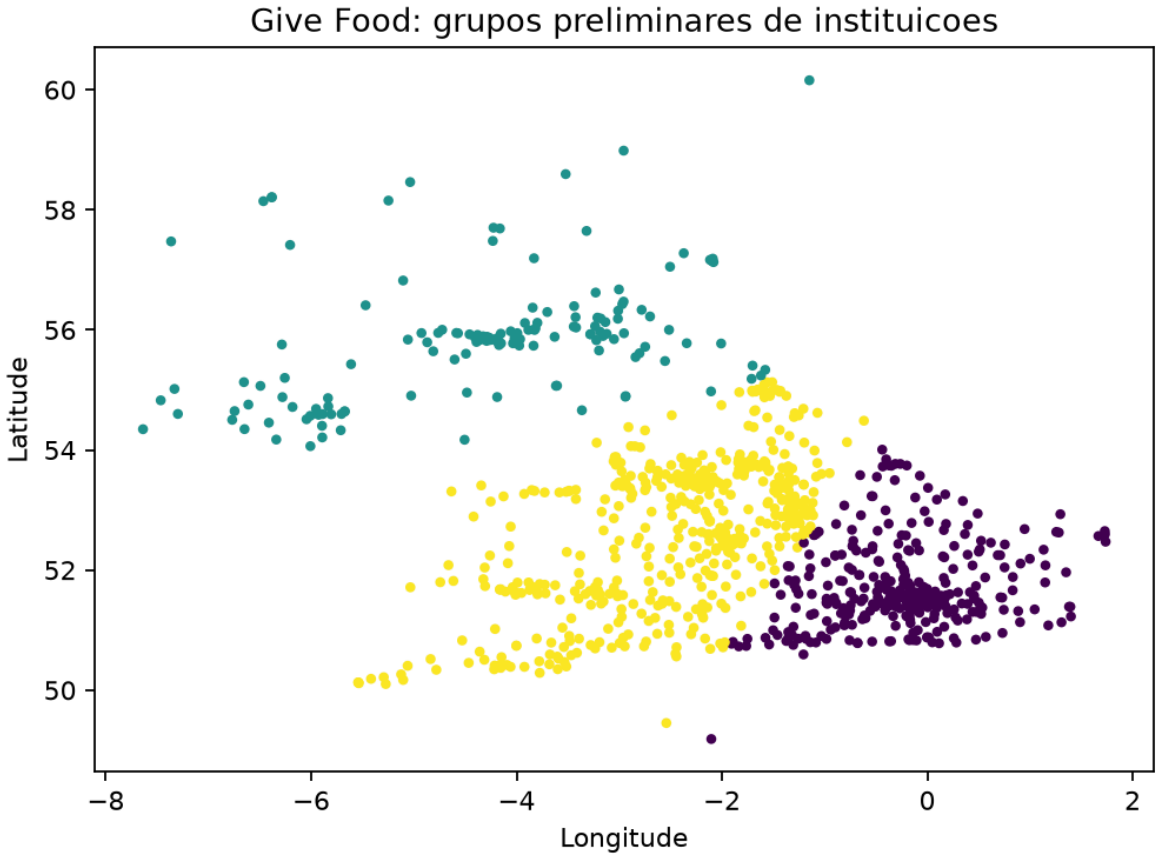


Figura 1 – Distribuição espacial das instituições conforme os grupos obtidos.
(Fonte: Elaborado a partir de `mining/outputs/instituicoes_clusters.csv`)

5. AVALIAÇÃO COMPARATIVA DOS AGRUPAMENTOS

A qualidade das partições espaciais geradas para cada valor de k foi avaliada de forma quantitativa através de três métricas oficiais: o Coeficiente de Silhueta

(coesão e separação), o Índice de Davies-Bouldin (sobreposição de grupos) e a Inércia (compactação interna).

Tabela 3 — Comparativo de desempenho dos agrupamentos (k de 2 a 6)

Qtd. Clusters (k)	Silhueta (Maior = Melhor)	Davies-Bouldin (Menor = Melhor)	Inércia (Compactação)
2	0,4969	0,7843	1097,12
3	0,4266	0,8546	696,80
4	0,4971	**0,6961**	432,75
5	0,4998	0,7252	356,64
6	**0,4998**	0,7252	356,64

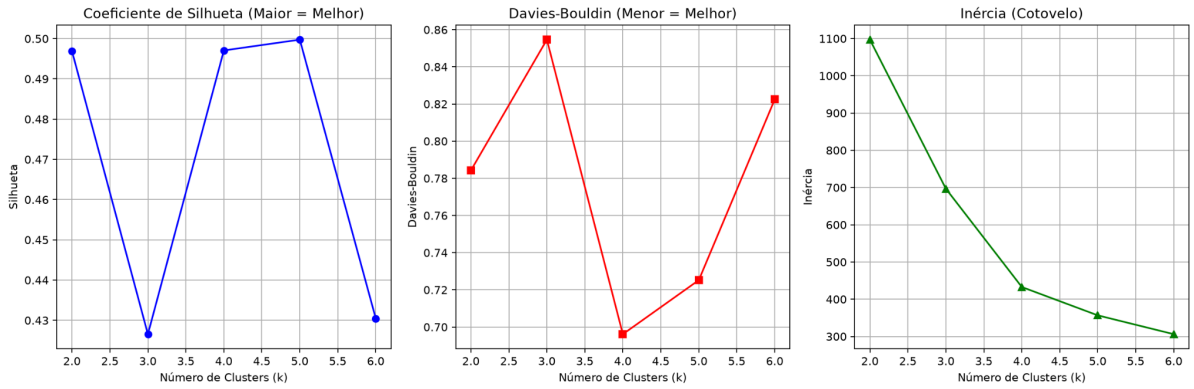


Figura 2 – Variação do Coeficiente de Silhueta, Índice de Davies-Bouldin e Inércia conforme o número de clusters (k)

Os resultados demonstrados na tabela evidenciam que as configurações com $k = 4$ e $k = 5$ otimizam estatisticamente a partição espacial. O valor $k = 4$ alcançou o menor índice de Davies-Bouldin (0,6961), indicando menor sobreposição entre os territórios delimitados, enquanto $k = 5$ registrou o ápice do Coeficiente de Silhueta (0,4998), assegurando alta coesão interna dos grupos formados.

6. LIMITES E CONTINUIDADE DO ESTUDO

6.1 RESTRIÇÕES METODOLÓGICAS E GEOGRÁFICAS

O agrupamento espacial fundamenta-se estritamente em coordenadas euclidianas planas obtidas de latitude e longitude. Essa abordagem desconsidera a

curvatura real da superfície terrestre (distâncias geodésicas reais), barreiras topográficas, malhas viárias e limites político-administrativos efetivos.

Ressalta-se que a concentração espacial de instituições em determinados clusters reflete exclusivamente a densidade de cadastros públicos disponíveis na base externa, não equivalendo a indicadores diretos de carência social, volume de doações arrecadadas ou prioridade operacional do sistema DoeFácil em Franca - SP.

6.2 PERSPECTIVAS DE EVOLUÇÃO

Em etapas futuras, recomenda-se a aplicação de algoritmos baseados em densidade (como DBSCAN) acoplados a métricas de distância esférica (Haversine). A transição para aprendizado supervisionado dependerá da consolidação de logs transacionais reais de estoque e movimentações no banco de dados relacional do aplicativo.

7. AMBIENTE E REPRODUÇÃO

O pipeline analítico foi validado em ambiente Windows com PowerShell, executando Python 3.15.0 isolado em ambiente virtual (.venv). As dependências essenciais encontramse travadas em versão no repositório.

Tabela 4 — Versões principais do ecossistema técnico

Biblioteca / Componente	**Versão Homologada**
Python	3.15.0
Pandas	3.0.6
Scikit-Learn	1.9.1
Matplotlib	3.11.2

Para replicar integralmente o processamento e a geração dos gráficos comparativos de métricas na raiz do repositório, utilize os comandos:

```
python -m venv .venv .\.venv\Scripts\Activate.ps1 pip install -r mining/requirements.txt python mining/tratar.py python mining/comparar_clusters.py
```

REFERÊNCIAS

GIVE FOOD. **API versão 2**. Disponível em: https://www.givefood.org.uk/api/2/. Acesso em: out. 2026.

PANDAS. **pandas.to_numeric Documentation (v3.0.6)**. Disponível em: https://pandas.pydata.org/. Acesso em: out. 2026.

SCIKIT-LEARN. **KMeans & Silhouette Score Documentation (v1.9.1)**. Disponível em: https://scikit-learn.org/. Acesso em: out. 2026

APÊNDICE A — CÓDIGO DE PREPARAÇÃO (TRATAR.PY)

```
from pathlib import Path
import hashlib, json
import pandas as pd
PASTA = Path(__file__).resolve().parent
FONTE = PASTA.parent / "givefoodbasededados.json"
saida = PASTA / "data"; saida.mkdir(exist_ok=True)

df = pd.json_normalize(json.loads(FONTE.read_text(encoding="utf-8")))
total = len(df)
df = df.drop_duplicates(subset="id").copy()
duplicados = total - len(df)

coordenadas = df["lat_lng"].str.split(",", n=1, expand=True)
df["latitude"] = pd.to_numeric(coordenadas[0], errors="coerce")
df["longitude"] = pd.to_numeric(coordenadas[1], errors="coerce")
df["name"] = df["name"].fillna("").str.strip()

validas = (df["closed"].eq(False) & df["name"].ne("") &
           df["latitude"].between(-90, 90) & df["longitude"].between(-180,
180))

limpo = df.loc[validas, ["id", "name", "country", "latitude", "longitude"]]
limpo.to_csv(saida / "instituicoes_tratadas.csv", index=False)

relatorio = {"registros_brutos": total, "duplicados_por_id": duplicados,
"registros_tratados": len(limpo)}
outputs = PASTA / "outputs"; outputs.mkdir(exist_ok=True)
(outputs / "tratamento.json").write_text(json.dumps(relatorio, indent=2,
ensure_ascii=False), encoding="utf-8")
```

APÊNDICE B — CÓDIGO DE COMPARAÇÃO DE CLUSTERS (COMPARAR_CLUSTERS.PY)

```
from pathlib import Path
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.preprocessing import StandardScaler

# Define o caminho base da pasta de mineração
PASTA = Path(__file__).resolve().parent

# Carrega a base de dados tratada gerada anteriormente
df = pd.read_csv(PASTA / "data" / "instituicoes_tratadas.csv")

# Prepara e padroniza as features geográficas (latitude e longitude)
X = StandardScaler().fit_transform(df[["latitude", "longitude"]])

if len(X) < 7:
    raise ValueError("A comparacao k=2..6 precisa de pelo menos 7 registros")

resultados = []

# Loop para testar diferentes quantidades de clusters (k de 2 a 6)
for k in range(2, 7):
    modelo = KMeans(n_clusters=k, random_state=42, n_init=10)
    grupos = modelo.fit_predict(X)

    if len(set(grupos)) != k:
        raise ValueError(f"Nao foram formados {k} grupos distintos")

    # Coleta as métricas de avaliação para cada k
    resultados.append({
        "k": k,
        "silhueta": float(silhouette_score(X, grupos)),
        "davies_bouldin": float(davies_bouldin_score(X, grupos)),
```

```
        "inercia": float(modelo.inertia_),
        "tamanhos": {str(i): int((grupos == i).sum()) for i in range(k)},
    })

# Cria a pasta de saída final se não existir
saida = PASTA / "outputs" / "final"
saida.mkdir(parents=True, exist_ok=True)

# Salva os resultados em formato JSON estruturado
(saida / "comparacao_clusters.json").write_text(
    json.dumps(resultados, ensure_ascii=False, indent=2), encoding="utf-8"
)

# Converte para DataFrame para facilitar a geração de gráficos
df_res = pd.DataFrame(resultados)

# Imprime o resumo em formato de tabela no terminal para conferência
print(df_res.drop(columns="tamanhos").to_string(index=False))

# --- GERAÇÃO E SALVAMENTO DOS GRÁFICOS ---
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Gráfico 1: Coeficiente de Silhueta
axes[0].plot(df_res["k"], df_res["silhueta"], marker="o", color="b",
linestyle="-")
axes[0].set_title("Coeficiente de Silhueta (Maior = Melhor)")
axes[0].set_xlabel("Número de Clusters (k)")
axes[0].set_ylabel("Silhueta")
axes[0].grid(True)

# Gráfico 2: Índice de Davies-Bouldin
axes[1].plot(df_res["k"], df_res["davies_bouldin"], marker="s", color="r",
linestyle="-")
axes[1].set_title("Davies-Bouldin (Menor = Melhor)")
axes[1].set_xlabel("Número de Clusters (k)")
axes[1].set_ylabel("Davies-Bouldin")
axes[1].grid(True)

# Gráfico 3: Inércia
axes[2].plot(df_res["k"], df_res["inercia"], marker="^", color="g",
linestyle="-")
```

```
axes[2].set_title("Inércia (Cotovelo)")
axes[2].set_xlabel("Número de Clusters (k)")
axes[2].set_ylabel("Inércia")
axes[2].grid(True)

plt.tight_layout()
plt.savefig(saida / "comparacao_metricas_clusters.png", dpi=160)
plt.close()

print(f"\nGráfico salvo com sucesso em: {saida /
'comparacao_metricas_clusters.png'}")
```